

Algoritmer & Datastrukturer
21. september 2026

Hashing

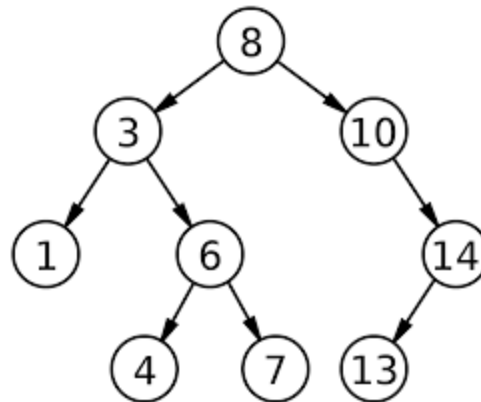
Hvor lang tid tager det at hente ting?

- Memory $\rightarrow O(1)$. 😊
- Sorteret tabel – logaritmisk tid $O(\log N)$.
- Ikke sorteret tabel – lineær tid $O(N)$.
- Linked list – lineær tid $O(N)$.
- I både stak og kø kan det gøres i konstant tid $O(1)$; men anvendelsesmulighederne er også stærkt begrænsede.
- Databaser: som oftest lynhurtigt, hvis man ved, hvad man leder efter; men alting har en pris.
- LLM's: neurale netværk; hurtige, men med en endnu højere pris.

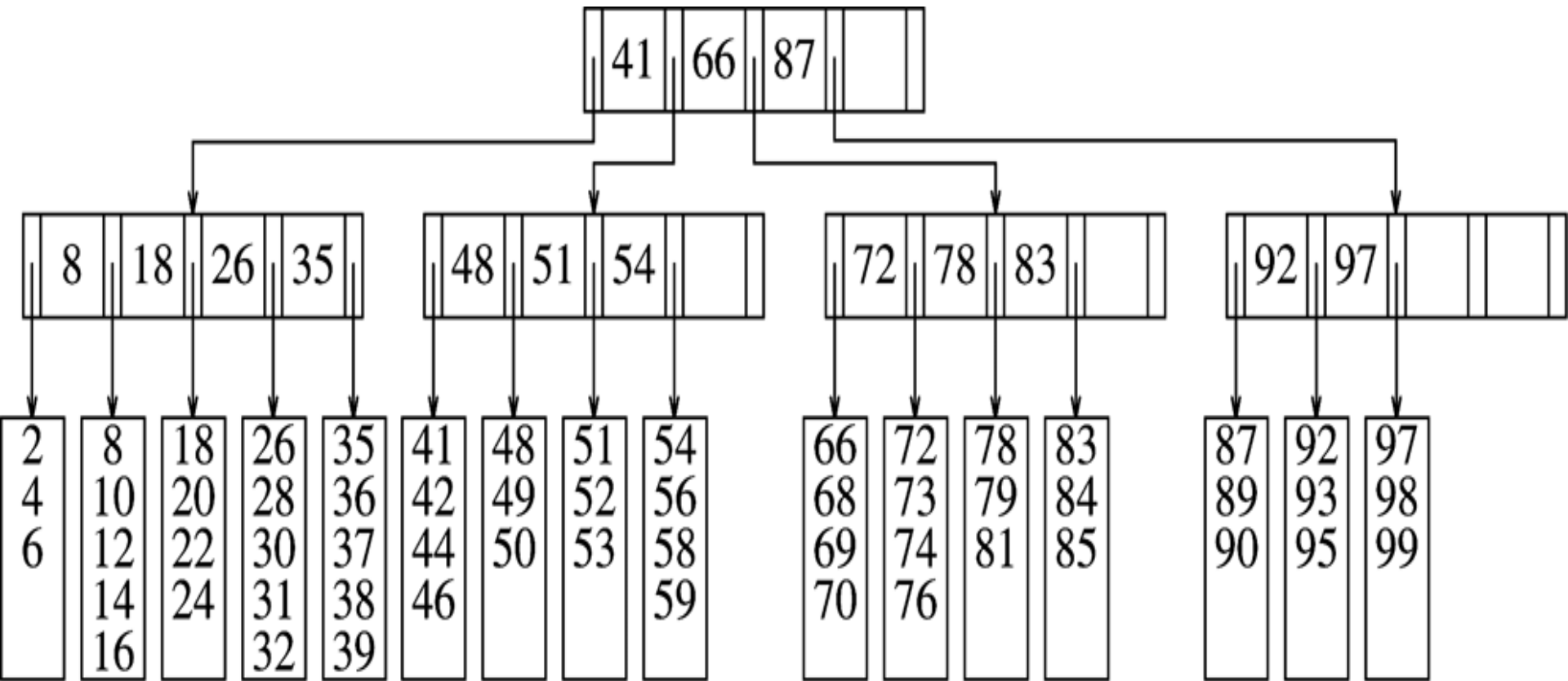
Hashing – kort fortalt

- Abstrakt datatype, hvor objekter kan hentes og gemmes i konstant tid – $O(1)$ – eller næsten.
- Kræver at hvert objekt har en éntydig nøgle; ellers er genfinding umulig.
- I traditionelle løsninger er objekterne lagret i et array hvis størrelse er et primtal.
- High performance kræver, at objekterne er ensartet distribueret i arrayet.
- Loadfaktoren betegnes som λ og er lig med antallet af elementer i tabellen divideret med tabellens størrelse og derfor altid $\leq 1,00$.
- I de fleste løsninger bør loadfaktoren være $< 0,50$.

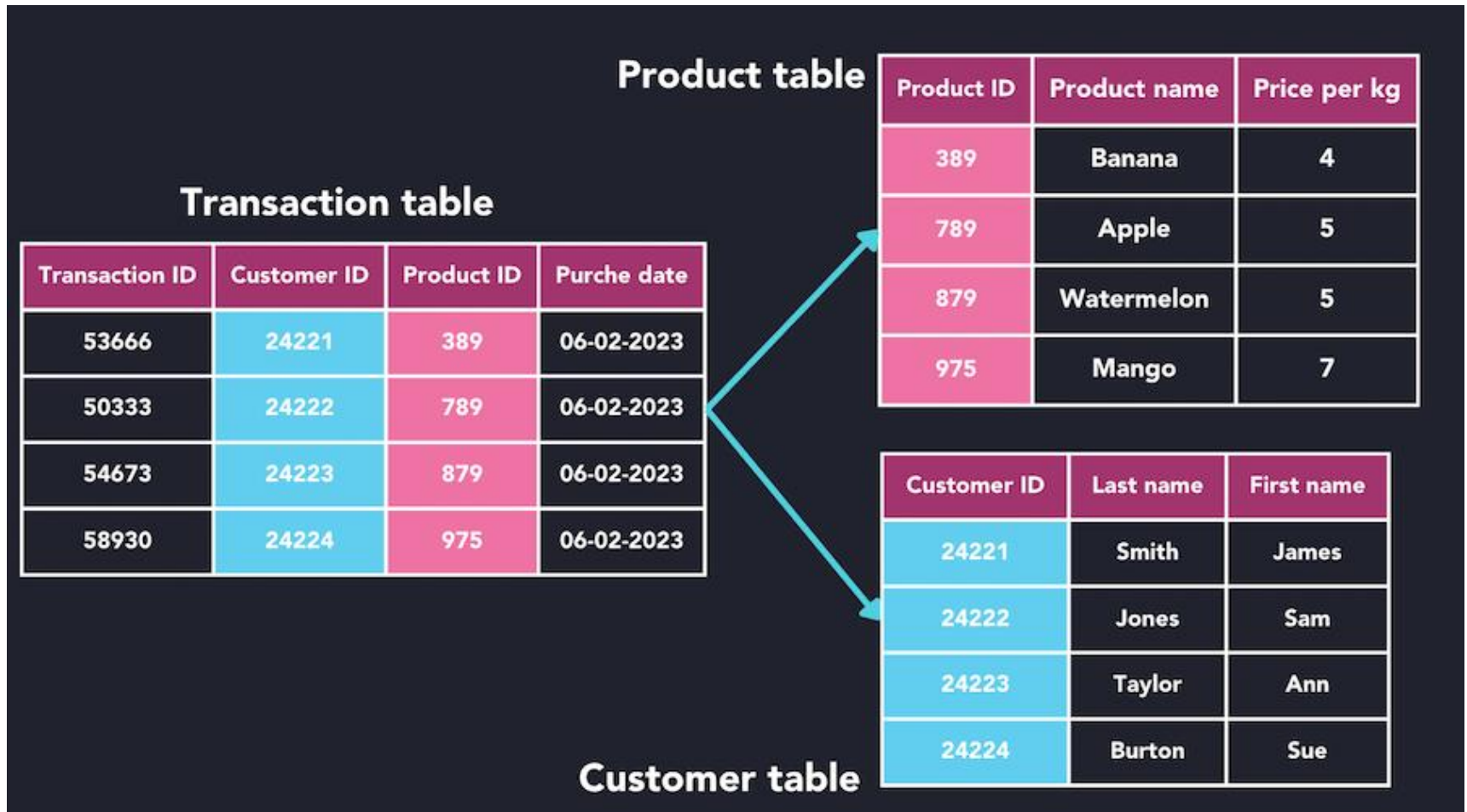
Sidespring 1 - Binært søgetræ



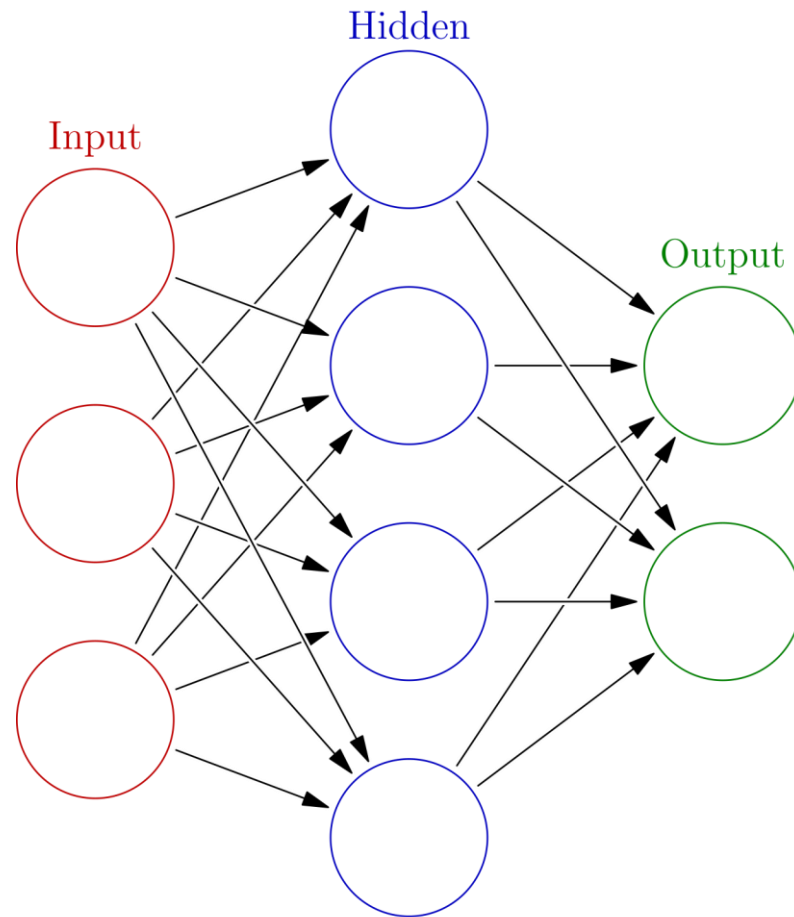
Sidespring 2 - 5-ordens B-træ



Sidespring 3 – Relational database



Sidespring 4 - Neuralt netværk



Hashing

... er et typisk eksempel på ***tradeoff***, hvilket betyder, at man får noget på bekostning af noget andet.

I dette tilfælde opnår man *fart* ved at ofre *plads*.

De fleste hashing implementeringer kræver, som nævnt, dobbelt så meget plads, som data for at opnå nogenlunde konstant tid ved indsættelse og læsning af objekter.

Hash Function

- Den mekanisme, som placerer et objekt i arrayet, kaldes en *hash funktion*.
- Der findes gode og dårlige hash funktioner.
- Eksempel, hvor nøglen er en `string`:

```
int hash( string key, int tableSize )
{
    int hashVal = 0;

    for( int i = 0; i < key.length( ); i++ )
        hashVal += key.charAt( i );

    return hashVal % tableSize;
}
```

- Hvad nu hvis nøglen har længden 8 og tabelstørrelsen er 10.007?
:

Simpel og effektiv hash funktion

```
int hash( string key, int tableSize )
{
    int hashVal = 0;

    for( int i = 0; i < key.length( ); i++ )
        hashVal = 37 * hashVal + key.charAt( i );

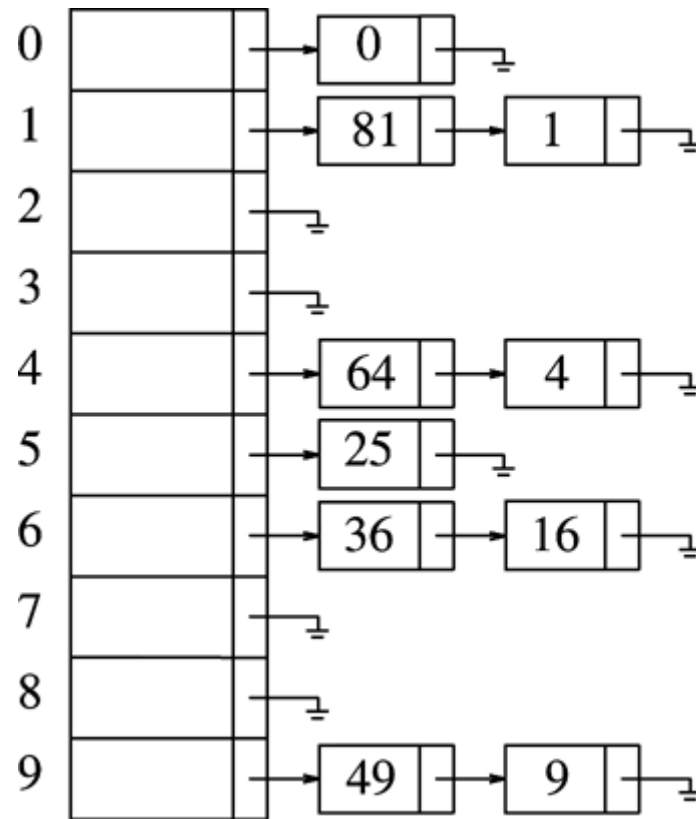
    hashVal %= tableSize;
    if( hashVal < 0 )
        hashVal += tableSize;

    return hashVal;
}
```

Hashings problem

- Uanset hvor god en hashfunktion man vælger, så vil det forekomme, at to eller flere objekter hasher til det samme indeks i tabellen.
- Dette kaldes en *kollision*.
- Kollisioner kan løses på forskellige måder:
 - Separate chaining
 - Linear probing
 - Quadratic probing
 - Double hashing
- Der er her tale om relativt simple strategier. Der findes andre, som er mere komplekse. Dem vender vi tilbage til.

Separate chaining – hash funktion % 10



Separate chaining

- Hvor meget plads har man brug for til listerne i separate chaining?
- Begrænset plads – her gælder reglen om double-up *ikke*.
- Hvor lange bliver disse lister under ‘normale’ omstændigheder?
- Disse problemer behandles i sektionerne 5.7 og 5.7.1.

Linear probing

- Ideen er, at når en kollision konstateres, placeres objektet på den næste ledige plads.
- Problemet med den løsning er, at der vil opstå *klyngedannelse* (clustering) – lidt lige som klynger af lastbiler på motorvejen.
- Fænomenet kaldes ofte *primary* clustering.
- Dette kan medføre uacceptable læse- og skrive-tider afhængig af loadfaktoren, λ .

Et eksempel på primary clustering ved linear probing og hash funktion $\% 10$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

Quadratic probing – hash funktion % 10 (~prøv næste kvadrattal)

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

Quadratic probing

Hvis *TableSize* er et primtal større end 3 og loadfaktoren er mindre end 0,5, så kan der **altid** findes en ledig plads, hvis der anvendes quadratic probing.

Double hashing

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89

$$\text{hash}_2(x) = R - (x \bmod R) \text{ for } R = 7$$

$$49 \rightarrow 9 + 7 - (49 \% 7) = 9 + (7 - 0) = 16 \rightarrow 6$$

$$58 \rightarrow 8 + 7 - (58 \% 7) = 8 + (7 - 2) = 13 \rightarrow 3$$

$$69 \rightarrow 9 + 7 - (69 \% 7) = 9 + (7 - 6) = 10 \rightarrow 0$$

$$60 \rightarrow 0 + 7 - (60 \% 7) = 9 + (7 - 4) = 13 \rightarrow 3 \rightarrow 6 \rightarrow 9 \rightarrow 2$$

Rehashing

- Når λ bliver for stor, skal tabellen udvides.
- Aktiviteten kaldes *rehashing*.
- Det kan gøres på forskellige måder
 - Når $\lambda \geq 0,5$.
 - Når en indsættelse fejler.
- Tabelstørrelsen fordobles
- Til det næste primtal (23 $\rightarrow$ 47).
- Rehashing er $O(N)$. Hvorfor?

Extendible hashing

Hvad skal man gøre, hvis der ikke er plads nok til hashtabellen i main memory?

EH – de originale data

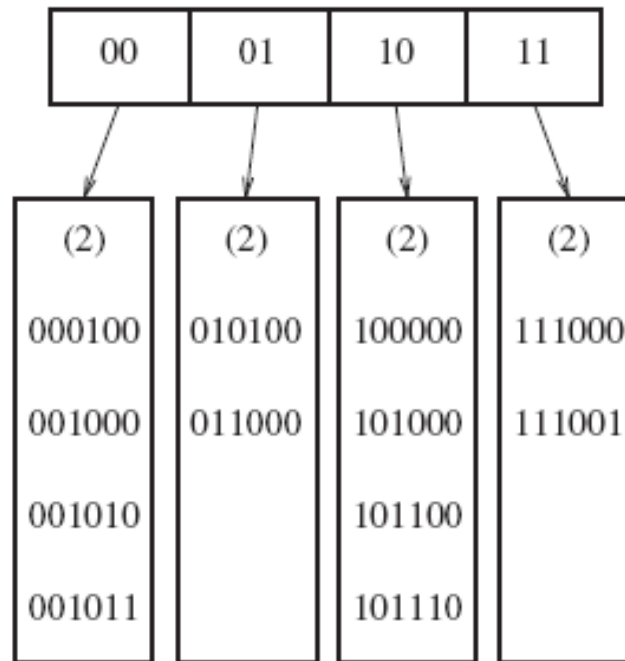


Figure 5.52 Extendible hashing: original data

EH – 100100 indsat

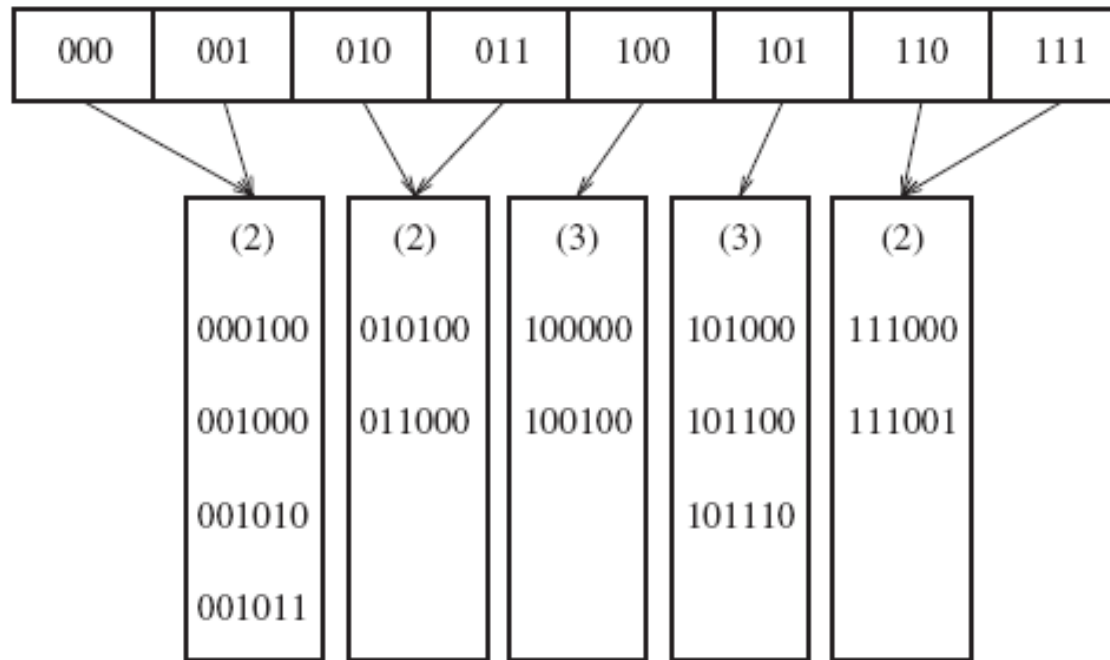


Figure 5.53 Extendible hashing: after insertion of 100100 and directory split

EH – 000000 indsat

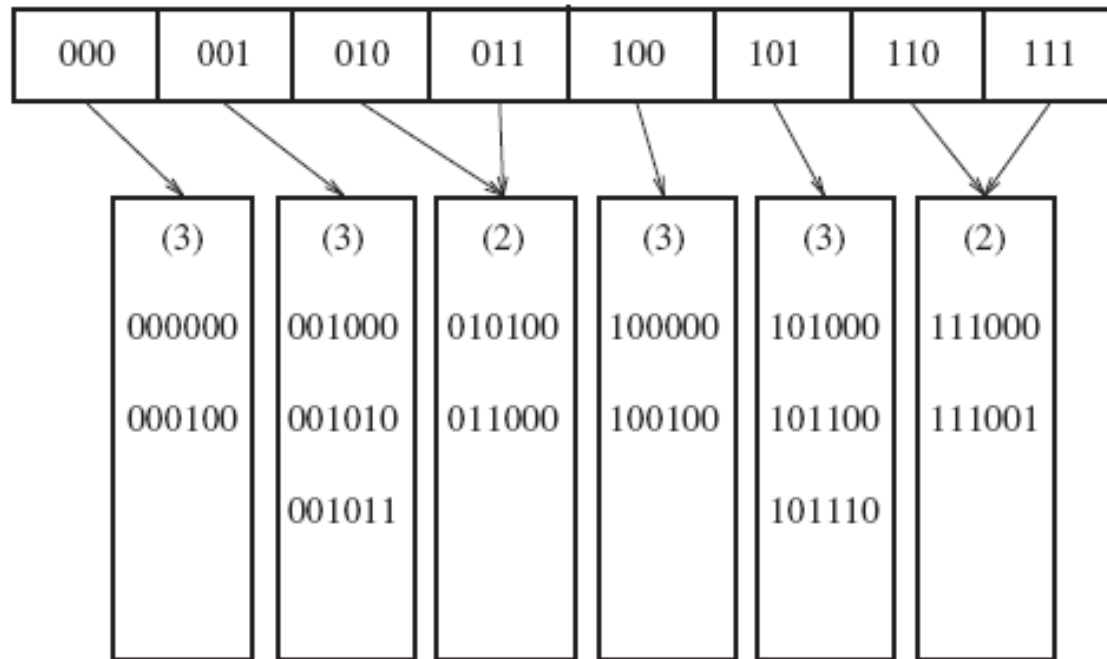


Figure 5.54 Extendible hashing: after insertion of 000000 and leaf split

Quote from a friend

” Extendible hashing is a dynamic hashing technique particularly suited to large, disk-based data structures where fast exact-key lookup is required and the data set changes in size without requiring the entire hash table to be reorganized.”

State-of-the-art hashing

Cuckoo-hashing

Hopscotch hashing

Dokumenter i mappen 'Ekstra Materialer',
på *itslearning*.

The power of two choices

Basic problem:

You have N (65,536) bins in which you wish to place N (65,536) balls.

How many balls do you need to make room for in each bin to avoid overflow?

Case 1: Single Random Choice

- Each ball picks one random bin
- Maximum load (with high probability):
- $L_1 \approx \ln(N) / \ln(\ln(N))$
- Example ($N = 65536$):
- $\ln(N) \approx 11.09$
- $\ln \ln(N) \approx 2.40$
- $L_1 \approx 4.6 \rightarrow$ fullest bin ≈ 4 – 5 balls

Case 2: Power of Two Choices

- Each ball picks two random bins
- Goes into the less loaded one
- Maximum load (with high probability):
 - $L_2 \approx \ln(\ln(N)) / \ln(2) + O(1)$
- Example ($N = 65536$):
 - $\ln \ln(N) \approx 2.40$
 - $\ln(2) \approx 0.693$
 - $L_2 \approx 3.5 \rightarrow$ fullest bin ≈ 3 –4 balls

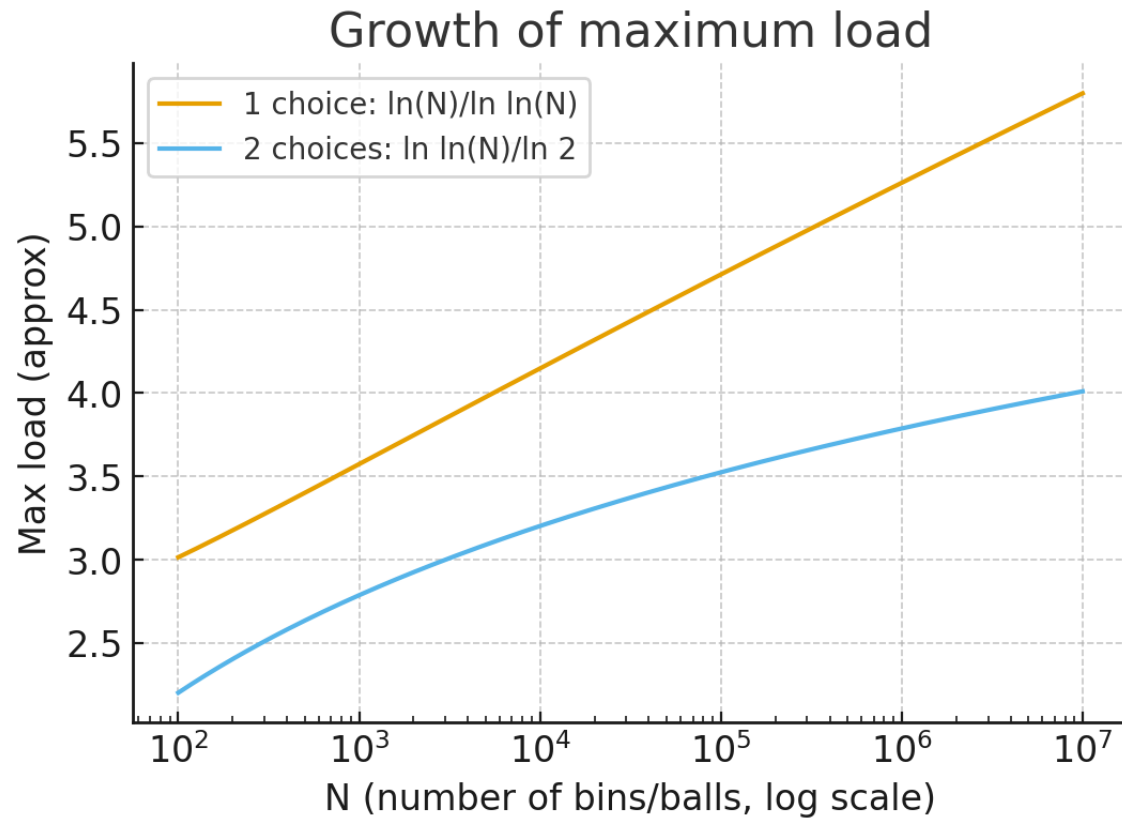
Key Takeaway

- 1 choice: max load $\sim \ln(N) / \ln \ln(N)$
- 2 choices: max load $\sim \ln \ln(N)$
-  As N grows:
- 1 choice $\rightarrow$ imbalance grows quickly
- 2 choices $\rightarrow$ imbalance grows extremely slowly

Load Balancing Example

- With $N = 65536$ servers:
- Random assignment (1 choice): busiest server $\approx 5\times$ average
- Two-choice assignment: busiest server $\approx 3\times$ average
- For millions of servers, improvement is dramatic!

Growth of Maximum Load



Cuckoo-hashing

- The power of two choices.
- To tabeller; to hash-funktioner (måske).
- Hvis pladsen er optaget, så smid beboeren ud (derfor navnet).
- Egenskaber:
 - Risiko for cykler; dog, meget lille, hvis $\lambda < 0,5$.
 - Det duer ikke, hvis $\lambda > 0,5$.
- Adskillige ændringer og forbedringer.
- Tilskrevet *Rasmus Pagh*, IT Univ., Kbh, 2004 (o.a.).

Hopscotch-hashing

- En forbedring af *linear probing*.
- I visse moderne arkitekturer er nærhed (proximity) vigtigere end antallet af forsøg.
- Stort set ikke behov for rehashing hvis $\lambda < 0,5$.
- Er Cuckoo-hashing overlegen på de fleste parametre.

Hopscotch eksempel

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1000
10	E	0000
11	G	1000
12	F	1000
13		0000
14		0000
...		

A: 7
B: 9
C: 6
D: 7
E: 8
F: 12
G: 11

Figure 5.46 Hopscotch hashing table. The hops tell which of the positions in the block are occupied with cells containing this hash value. Thus $\text{Hop}[8] = 0010$ indicates that only position 10 currently contains items whose hash value is 8, while positions 8, 9, and 11 do not

Hopscotch eksempel

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1000
10	E	0000
11	G	1000
12	F	1000
13		0000
14		0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1000
10	E	0000
11		0010
12	F	1000
13	G	0000
14		0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1010
10	E	0000
11	H	0010
12	F	1000
13	G	0000
14		0000
...		

A: 7
B: 9
C: 6
D: 7
E: 8
F: 12
G: 11
H: 9

Figure 5.47 Hopscotch hashing table. Attempting to insert *H*. Linear probing suggests location 13, but that is too far, so we evict *G* from position 11 to find a closer position

Øvelser – kan løses i hånden

1

En hashtabel har plads til 16 elementer.

Indsæt fem elementer, som alle hasher til samme position i tabellen med quadratic probing.

2

En hashtabel har plads til 11 elementer.

Indsæt seks elementer, som alle hasher til samme position i tabellen med quadratic probing.

3

Hvordan skal man håndtere *sletninger*, hvis der anvendes probing/open addressing?

Øvelser

Dokument på *itslearning* med flere øvelser.

Vigtigste slides

2

3

8

17

19